Lab 5 Static Code Analysis

Ronak Thamaran – PES2UG24CS410
G (CSE)

Issues Identified and Fixed

Issue	Tool / type	Description	Fix applied
Mutable default argument	Pylint W0102 / bug	A shared list such as logs=[] can retain data between calls.	Use None as the default and create state inside the method; the final implementation stores state safely in self._items.
Overly broad exception	Pylint W0718 / robustness	A bare or broad exception hides programming errors and makes failures hard to diagnose.	Validate inputs before processing and raise specific ValueError messages.
Unsafe dynamic execution	Bandit B307 / security	Using eval on user-controlled input can execute unintended code.	No eval, exec, shell command, or unsafe deserialization is used; input is checked as ordinary data.
Line-length style violation	Flake8 E501 / style	Lines longer than 79 characters reduce readability and fail PEP 8 checks.	Long statements were wrapped and the final Flake8 report is empty.

Reflection

1. Which issues were easiest and hardest to fix?

The easiest issue was the line-length violation because it only required rewrapping the statement. Input validation was more involved because each operation had to define valid names, integer quantities, non-negative values, and insufficient-stock behavior consistently.

2. Did the tools report any false positives?

No false positives were reported in the final run. Static-analysis findings still need review in real projects: for example, a security rule may flag a function name even when the input is trusted and constrained. The reviewer should document why any deliberately retained finding is safe.

3. How would you integrate static analysis into a workflow?

I would run Flake8 and Pylint locally before each commit, then run all three tools in continuous integration for every pull request. A project-specific configuration file would keep rules consistent, and high-severity Bandit findings would block merging until reviewed.

4. What tangible improvements resulted?

The final code is clearer because each method has one purpose and explicit error messages. It is more robust because invalid names, invalid quantities, and attempts to remove unavailable stock are rejected. It is safer because it does not evaluate user input or use risky execution patterns.

Final Static Analysis Results

Pylint output

```
--------------------------------------------------------
Your code has been rated at 10.00/10
```

Bandit output

```
[main]■INFO■profile include tests: None
[main]■INFO■profile exclude tests: None
[main]■INFO■cli include tests: None
[main]■INFO■cli exclude tests: None
[main]■INFO■running on Python 3.12.14
Run started:2026-10-16 11:13:24.933756+00:00

Test results:
■No issues identified.

Code scanned:
■Total lines of code: 54
■Total lines skipped (#nosec): 0
■Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
■Total issues (by severity):
■■Undefined: 0
■■Low: 0
■■Medium: 0
■■High: 0
■Total issues (by confidence):
■■Undefined: 0
■■Low: 0
■■Medium: 0
■■High: 0
Files skipped (0):
```

Flake8 output

```
No violations reported.
```

Result summary: Pylint score 10.00/10; Bandit reported no issues; Flake8 reported no violations.